

Управляемый кластер Kubernetes

Kubernetes полезен для команды, для проекта, упрощает развертывание, масштабируемость, устойчивость, он позволяет нам использовать любую базовую инфраструктуру

Содержание

- Создание / упаковка / запуск приложений ReactJS, Java и Python
- Контейнеры Docker; как определять и создавать их с помощью Dockerfiles,
- Реестры контейнеров; мы использовали Docker Hub в качестве хранилища для наших контейнеров.

Наиболее важные части Kubernetes.

- Модули
- Услуги
- Развертывания
- Новые концепции, такие как развертывание с нулевым временем простоя
- Создание масштабируемых приложений

Приложение Анализ Эмоциональности

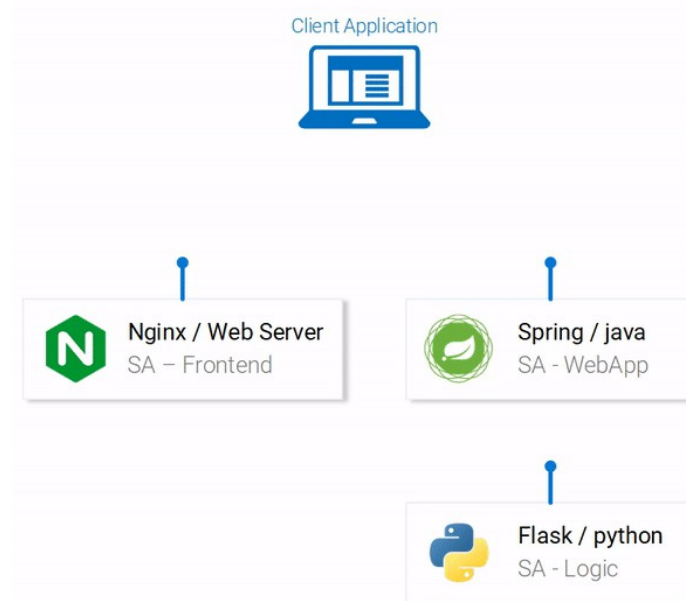
Приложение состоит из трех микросервисов. Каждый из них обладает определенной функциональностью:

- **SA-интерфейс:** веб-сервер Nginx, который **обслуживает** статические файлы ReactJS.
- **SA-WebApp:** веб-приложение Java, которое **обрабатывает запросы** от интерфейса.
- **SA-Logic:** приложение на Python, которое **выполняет анализ настроений**.
- Важно знать, что микросервисы не живут изолированно, они обеспечивают “разделение задач”, но им все равно **приходится** взаимодействовать друг с другом.

Sentiment Analyser

Type your sentence.

SEND



Комментарий

Взаимодействие сервисов иллюстрируется тем, как данные передаются между ними:

- Клиентское приложение запрашивает index.html (которое, в свою очередь, запрашивает связанные скрипты приложения ReactJS)
- Пользователь, взаимодействующий с приложением, запускает запросы к веб-приложению Spring.
- Веб-приложение Spring пересылает запросы на анализ настроек в приложение Python.
- Приложение Python вычисляет настройку и возвращает результат в качестве ответа.
- Веб-приложение Spring возвращает ответ приложению React. (Который затем представляет информацию пользователю.)

Код для всех этих приложений можно найти в [этом репозитории](#).

Настройка React для локальной разработки

- Для запуска приложения React на вашем компьютере должны быть установлены NodeJS и NPM. После их установки перейдите с помощью своего терминала в каталог sa-frontend. Введите следующую команду:
- `npm install`
- Это загружает все зависимости Javascript приложения React и помещает их в папку `node_modules`. (Зависимости определены в файле `package.json`). После устранения всех зависимостей выполните следующую команду:
- `npm start`
- Так запускается приложение react, и по умолчанию вы можете получить к нему доступ на `localhost:3000`.

Подготовка приложения React к работе в сети

- Для производства нам нужно встроить наше приложение в статические файлы и обслуживать их с помощью веб-сервера.
- Чтобы создать приложение React, перейдите в своем терминале к каталогу sa-frontend . Затем выполните следующую команду:
- `npm run build`
- В дереве вашего проекта будет создана папка с именем build. Эта папка содержит все статические файлы, необходимые для нашего приложения ReactJS.

Обслуживание статических файлов с помощью Nginx

- Установите и запустите веб-сервер Nginx ([как](#)). Затем переместите содержимое папки sa-frontend / build в `[your_nginx_installation_dir]/html`.
- Таким образом, сгенерированный index.html файл будет доступен в `[your_nginx_installation_dir]/html/index.html` . Это **его файл по умолчанию, который обслуживает Nginx**.
- По умолчанию веб-сервер Nginx прослушивает порт 80. Вы можете указать другой порт, обновив свойство `server.listen` в файле `[your_nginx_installation_dir]/conf/nginx.conf`.
- Откройте свой браузер и нажмите на конечную точку localhost: 80, увидите, как появится приложение ReactJS.

Настройка веб-приложения Spring

Для запуска приложения Spring у вас должны быть установлены JDK8 и Maven. (их переменные среды также должны быть настроены). После их установки вы можете перейти к следующей части.

Упаковка приложения в Jar

- Перейдите в своем терминале в каталог sa-webapp и введите следующую команду:
- `mvn install`
- При этом будет создана папка с именем target в каталоге sa-webapp. В папке target у нас есть наше Java-приложение, упакованное в виде jar: 'sentiment-analysis-web-0.0.1-SNAPSHOT.jar'

Определение свойства

В Spring источником свойств по умолчанию является application.properties. (Находится в sa-webapp/src/main/resources). Но это не единственный способ определения свойства.

Свойство должно быть инициализировано значением, определяющим, где выполняется наше приложение Python, таким образом, мы сообщим нашему веб-приложению Spring, куда пересылать сообщения во время выполнения.

- Чтобы упростить задачу, будем запускать приложение Python на localhost:5000.
- Выполните приведенную ниже команду, и можно перейти к последнему сервису - приложению python.
- `java -jar sentiment-analysis-web-0.0.1-SNAPSHOT.jar`
- `--sa.logic.api.url=http://localhost:5000`

Настройка приложения на Python

Для запуска приложения на Python нам необходимо установить Python3 и Pip. (Их переменные среды также должны быть настроены).

- Установка зависимостей
- Перейдите в терминале в каталог sa-logic/sa (repo) и введите следующую команду:
- `python -m pip install -r requirements.txt`
- `python -m textblob.download_corpora`

Запуск приложения

После использования Pip для установки зависимостей мы готовы запустить приложение, выполнив следующую команду:

- `python sentiment_analysis.py`
- * Running on `http://0.0.0.0:5000/` (Press CTRL+C to quit)

Это означает, что наше приложение запущено и прослушивает HTTP-запросы на порту 5000 на localhost.

Проверка кода Flask

Давайте исследуем код, чтобы понять, что происходит в приложении Python с SA Logic .

- `from textblob import Text Blob`
- `from flask import Flask, request, jsonify`
- `app = Flask(__name__)` #1
- `@app.route("/analyse/sentiment", methods=['POST'])` #2
- `def analyse_sentiment():`
- `sentence = request.get_json()['sentence']` #3
- `polarity = TextBlob(sentence).sentences[0].polarity` #4
- `return jsonify(` #5
- `sentence=sentence,`
- `polarity=polarity`
- `)`
- `if __name__ == '__main__':`
- `app.run(host='0.0.0.0', port=5000)` #6

1. Создайте экземпляр объекта Flask.
2. Определяет путь, по которому может быть отправлен запрос POST.
3. Извлеките свойство “предложение” из тела запроса.
4. Создайте экземпляр анонимного объекта TextBlob и получите полярность из первого предложения. (У нас есть только один).
5. Возвращайте вызывающему ответ с текстом, содержащим предложение и полярность.
6. Запустите приложение flask object для прослушивания запросов на 0.0.0.0:5000 (вызовы на localhost: 5000 также будут поступать в это приложение).

Службы настроены для взаимодействия друг с другом. Повторно откройте интерфейс в localhost: 80 и попробуйте их, прежде чем продолжить!

Создание образов контейнеров для каждой службы

Рассмотрим, как запускать службы в контейнерах Docker, поскольку это является необходимым условием для их запуска в кластере Kubernetes.

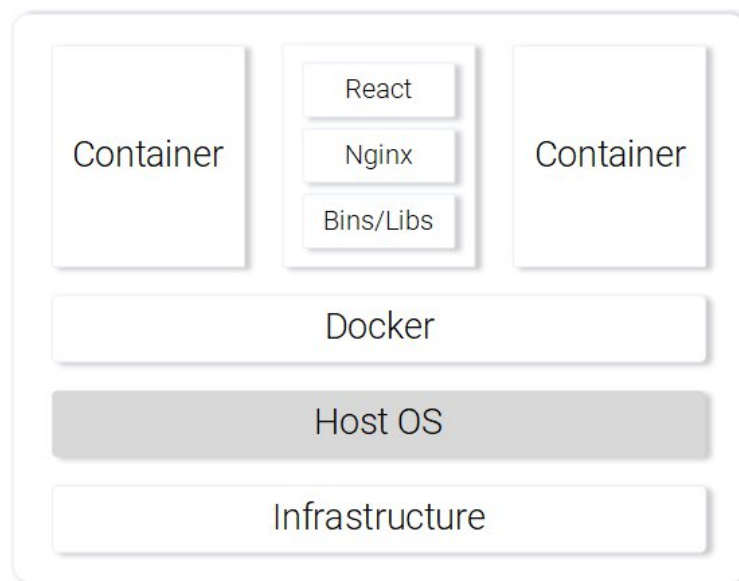
Kubernetes - это инструмент для организации контейнеров. Понятно, что нам нужны контейнеры, чтобы иметь возможность их организовывать. Но что такое контейнеры? На этот вопрос лучше всего ответить из документации в docker.

- Образ контейнера - это легкий, автономный, исполняемый пакет программного обеспечения, который включает в себя все необходимое для его запуска: код, среду выполнения, системные инструменты, системные библиотеки, настройки.
- Контейнерное программное обеспечение, доступное как для приложений на базе Linux, так и для Windows, всегда будет работать одинаково, независимо от среды.
- Это означает, что контейнеры могут запускаться на любом компьютере — даже на рабочем сервере — без каких-либо различий.

Отправка статических файлов React из контейнера

Плюсы использования контейнера.

- Экономьте ресурсы, используйте ОС хоста с помощью Docker.
- Не зависит от платформы. Контейнер, который вы запускаете на своем компьютере, будет работать где угодно.
- Облегченный интерфейс с использованием слоев изображений.



Веб-сервер Nginx, обслуживающий статические файлы в контейнере

Создание образа контейнера для приложения React (вводная часть Docker)

Основным строительным блоком для контейнера Docker является файл `.dockerfile`. Dockerfile начинается с базового образа контейнера и сопровождается последовательностью инструкций о том, как создать новый образ контейнера, соответствующий потребностям вашего приложения.

Прежде вспомним шаги, которые мы предприняли для обслуживания статических файлов react с помощью nginx:

- Создайте статические файлы (`npm run build`)
- Запустите сервер nginx
- Скопируйте содержимое папки `build` из вашего `sa-frontend` проекта в `nginx/html`.

Определение файла Dockerfile для SA-интерфейса

Инструкции в Dockerfile для SA-интерфейса - это всего лишь двухэтапная задача. Это потому, что команда Nginx предоставила нам базовый образ для Nginx, который мы будем использовать для сборки поверх. Два шага:

- Начните с базового образа Nginx
- Скопируйте каталог `sa-frontend/build` в каталог контейнеров `nginx/html`.

Преобразованный в файл Dockerfile, он выглядит как:

- FROM nginx
- COPY build /usr/share/nginx/html

Разве это не удивительно, это даже доступно для чтения, давайте резюмируем:

- Начните с образа nginx. Скопируйте каталог сборки в каталог `nginx/html` на изображении. Вот и все!
- Возможно, вам интересно, как я узнал, куда копировать файлы сборки? т. е. `/usr/share/nginx/html`. Довольно просто: это было задокументировано в образе nginx в Docker Hub.

Создание и запуск контейнера

Прежде чем мы сможем запустить наше изображение, нам нужен реестр контейнеров для размещения наших изображений.

Docker Hub - это бесплатный облачный контейнерный сервис, который мы будем использовать для этой демонстрации. Перед продолжением вам предстоит выполнить три задачи:

- Установите Docker CE
- Зарегистрируйтесь в Docker Hub.

Войдите в систему, выполнив приведенную ниже команду в вашем терминале:

- `docker login -u="$DOCKER_USERNAME" -p="$DOCKER_PASSWORD"`

После выполнения вышеуказанных задач перейдите в каталог sa-frontend. Затем выполните приведенную ниже команду (замените \$DOCKER_USER_ID на ваше имя пользователя docker hub. Например, для rinormaloku/ sentiment-analysis-frontend)

- `docker build -f Dockerfile -t $DOCKER_USER_ID/sentiment-analysis-frontend .`

Чтобы отправить изображение, используйте команду `docker push`:

- `docker push $DOCKER_USER_ID/sentiment-analysis-frontend`

Убедитесь в своем репозитории docker hub, что изображение было успешно загружено.

Запуск контейнера

Теперь изображение в \$DOCKER_USER_ID/sentiment-analysis-frontend может быть извлечено и запущено кем угодно:

```
docker pull $DOCKER_USER_ID/sentiment-analysis-frontend
```

```
docker run -d -p 80:80 $DOCKER_USER_ID/sentiment-analysis-frontend
```

Контейнер Docker запущен!

Создание образа контейнера для приложения Java

Откройте файл Dockerfile в sa-webapp, и вы найдете только два новых ключевых слова:

- ENV SA_LOGIC_API_URL http://localhost:5000
- ...
- EXPOSE 8080

Ключевое слово ENV объявляет переменную среды внутри контейнера docker. Это позволит нам предоставлять URL для API анализа настроек при запуске контейнера.

Кроме того, ключевое слово EXPOSE предоставляет доступ к порту, к которому мы хотим получить доступ позже. Это только для целей документации, другими словами, это будет служить информацией для человека, читающего файл Dockerfile.

Вы должны быть знакомы с созданием и использованием образа контейнера. Если возникнут какие-либо трудности, прочитайте README.md файл в каталоге sa-webapp.

Создание образа контейнера для приложения Python

В файле Dockerfile в sa-logic нет новых ключевых слов.

- Создание контейнера Docker

```
$ docker build -f Dockerfile -t $DOCKER_USER_ID/sentiment-analysis-logic .
```

- Запуск контейнера Docker

```
$ docker run -d -p 5050:5000 $DOCKER_USER_ID/sentiment-analysis-logic
```

Приложение по умолчанию прослушивает порт 5000. Порт 5050 хост-машины сопоставлен с портом 5000 контейнера.

-страница 5050:5000 т.е.

-p <hostPort>:<containerPort>

Тестирование контейнерного приложения

Протестируем эти контейнеры.

Запустите контейнер sa-logic и настройте прослушивание на порту 5050:

- `docker run -d -p 5050:5000 $DOCKER_USER_ID/sentiment-analysis-logic`

Запустите sa-webapp container и настройте прослушивание на порту 8080, и дополнительно нам нужно изменить порт, на котором прослушивается приложение python, переопределив переменную среды SA_LOGIC_API_URL.

- `$ docker run -d -p 8080:8080 -e SA_LOGIC_API_URL='http://<container_ip or docker machine ip>:5000' $DOCKER_USER_ID/sentiment-analysis-web-app`

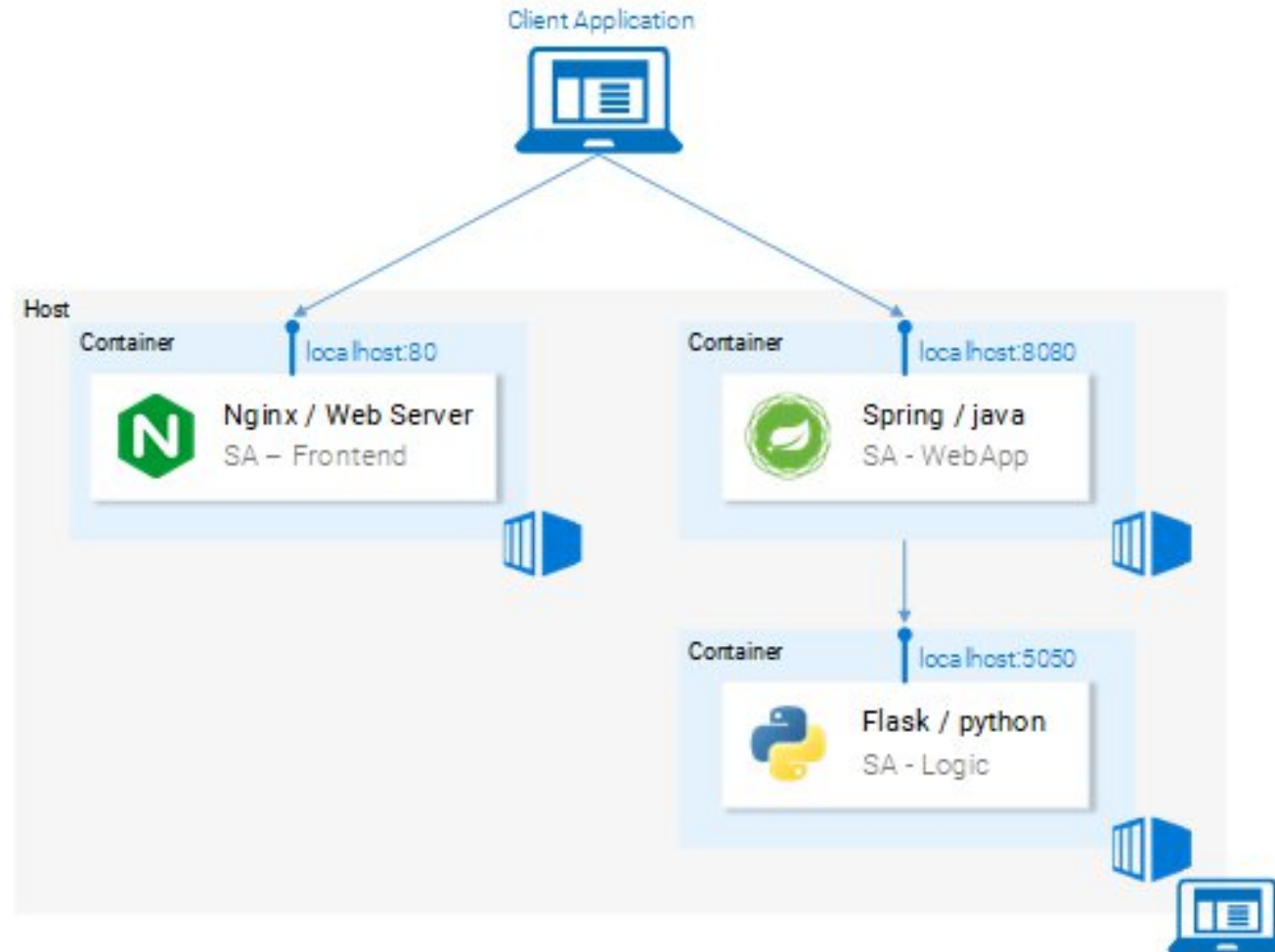
Ознакомьтесь с README о том, как получить IP контейнера или docker machine ip.

Запустите sa-frontend container:

- `docker run -d -p 80:80 $DOCKER_USER_ID/sentiment-analysis-frontend`

Откройте свой браузер на localhost:80.

Микросервисы работающие на контейнерах



Что такое Kubernetes

После того, как мы запустили наши микросервисы из контейнеров, у нас возник один вопрос, давайте подробнее рассмотрим его в формате вопросов и ответов:

- Вопрос: Как мы масштабируем контейнеры?
- А: Мы запускаем еще один.
- Вопрос: Как мы распределяем нагрузку между ними? Что, если сервер уже используется по максимуму и нам нужен другой сервер для нашего контейнера? Как нам рассчитать наилучшее использование оборудования?
- А: Хм... Эрмм...
- Вопрос: выпуск обновлений без каких-либо нарушений? И если мы это сделаем, как мы можем вернуться к рабочей версии.

“Kubernetes - это контейнерный оркестратор, который абстрагирует базовую инфраструктуру. (Где запускаются контейнеры)”.

Абстрагирование базовой инфраструктуры

Kubernetes абстрагирует базовую инфраструктуру, предоставляя нам простой API, к которому мы можем отправлять запросы.

Эти запросы побуждают Kubernetes выполнять их наилучшим образом.

Например, это так же просто, как запросить “Kubernetes раскрутить 4 контейнера изображения x”.

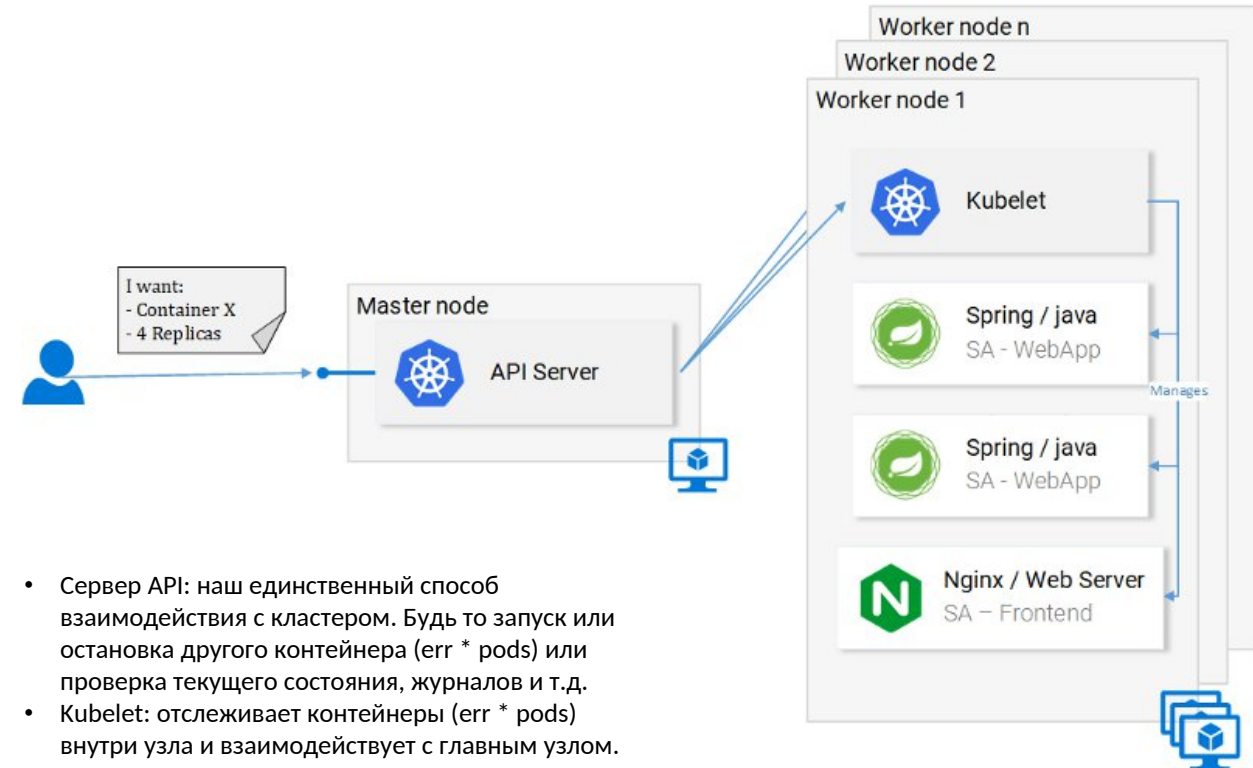
Затем Kubernetes найдет недостаточно используемые узлы, на которых он будет разворачивать новые контейнеры (см. рис.).

Что это значит для разработчика?

Что ему не нужно заботиться о количестве узлов, где запускаются контейнеры и как они взаимодействуют.

Он не занимается оптимизацией оборудования и не беспокоится о том, что узлы выйдут из строя (а они выйдут из строя по закону Мерфи), потому что в кластер Kubernetes могут быть добавлены новые узлы.

Тем временем Kubernetes развернет контейнеры на других узлах, которые все еще работают. Он делает это с наилучшими возможными возможностями.



- Сервер API: наш единственный способ взаимодействия с кластером. Будь то запуск или остановка другого контейнера (err * pods) или проверка текущего состояния, журналов и т.д.
- Kubelet: отслеживает контейнеры (err * pods) внутри узла и взаимодействует с главным узлом.
- * Модули: изначально думайте о модулях как о контейнерах.

Стандартизация поставщиков облачных услуг

Еще одним преимуществом Kubernetes является то, что он стандартизирует поставщиков облачных услуг (CSP). Давайте подробнее рассмотрим это на примере:

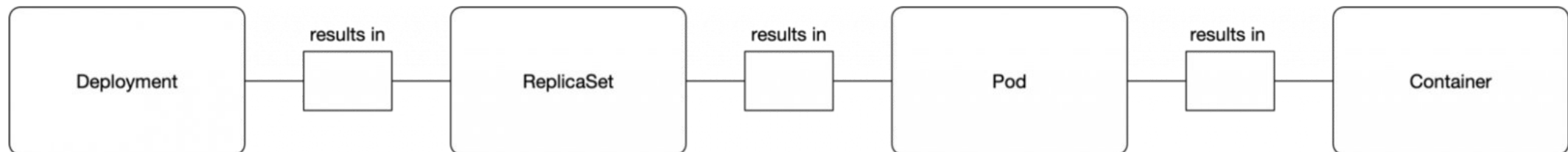
– Эксперт в Azure, Google Cloud Platform или каком-либо другом CSP заканчивает работу над проектом в совершенно новом CSP, и у него нет опыта работы с ним. Это может иметь множество последствий, и это лишь некоторые из них: он может пропустить крайний срок; компании, возможно, потребуется нанять больше ресурсов и так далее.

В отличие от этого, с Kubernetes это вообще не проблема. Потому что вы будете выполнять одни и те же команды на сервере API независимо от того, какой CSP. Вы в декларативной форме запрашиваете у сервера API то, что вы хотите. Kubernetes абстрагируется и реализует как для рассматриваемого CSP.

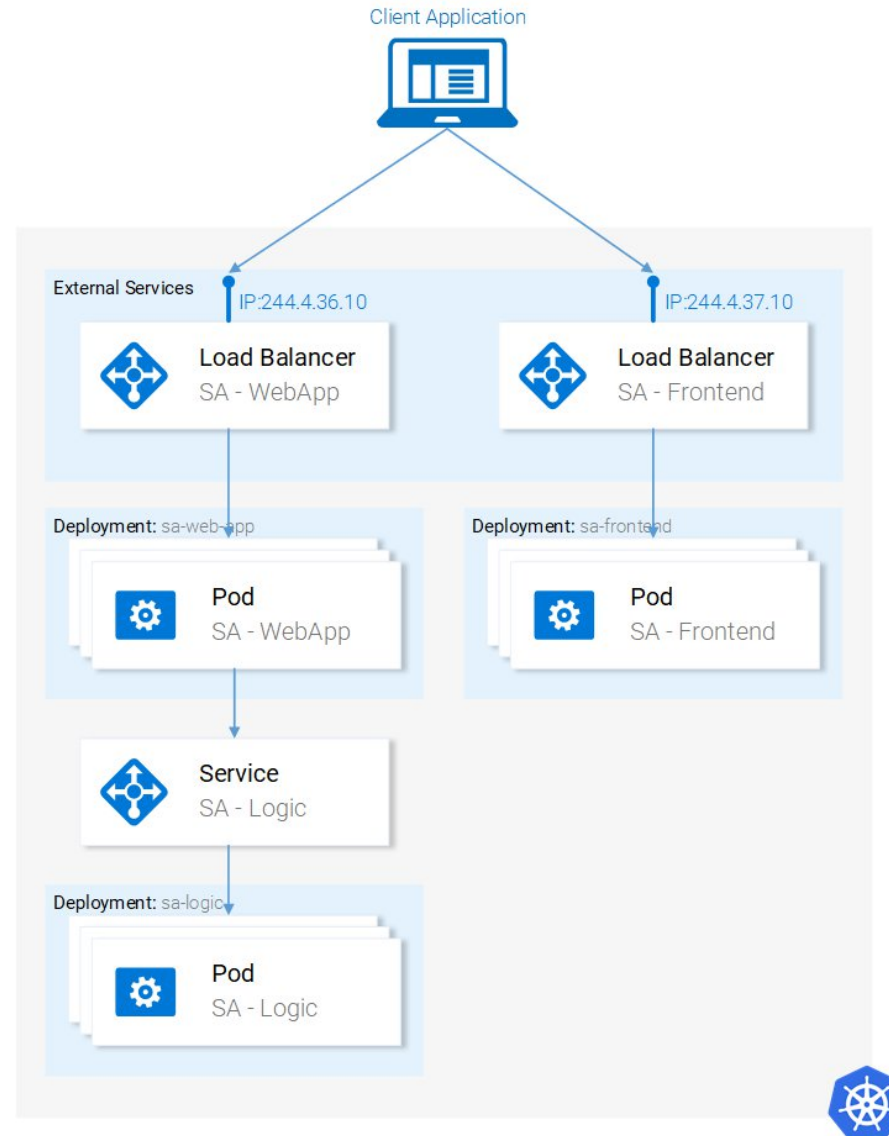
Дайте ему секунду, чтобы вникнуть — это чрезвычайно мощная функция. Для компании это означает, что они не привязаны к CSP. Они подсчитывают свои расходы на другой CSP и двигаются дальше. У них все еще будет опыт, у них все еще будут ресурсы, и они могут сделать это дешевле!

Kubernetes — движок оркестровки контейнеров

- Pods (Поды) — базовые строительные блоки Kubernetes.
- Pod представляет собой запрос на запуск одного или более контейнеров на одном узле.
- Pod определяется представлением запроса на запуск (execute) одного или более контейнеров на одном узле, и эти контейнеры разделяют доступ к таким ресурсам, как тома хранилища и сетевой стек.
- Pod — это **один и единственный** объект в Kubernetes, который приводит к запуску контейнеров. **Нет pod'а — нет контейнера!**



Микросервисы, работающие в кластере, управляемом Kubernetes



Установка и запуск Minikube

Следуйте официальной документации для установки Minikube. Во время установки Minikube вы также установите Kubectl. Это клиент для выполнения запросов к серверу API Kubernetes.

Чтобы запустить Minikube, выполните команду `minikube start`, а после ее завершения выполните `kubectl get nodes`, и вы должны получить следующий результат

- `kubectl get nodes`
- | NAME | STATUS | ROLES | AGE | VERSION |
|----------|--------|--------|-----|---------|
| minikube | Ready | <none> | 11m | v1.9.0 |

Minikube предоставляет нам кластер Kubernetes, который имеет только один узел, но помните, что нам все равно, сколько там узлов, Kubernetes абстрагирует это, и для изучения Kubernetes не имеет значения.

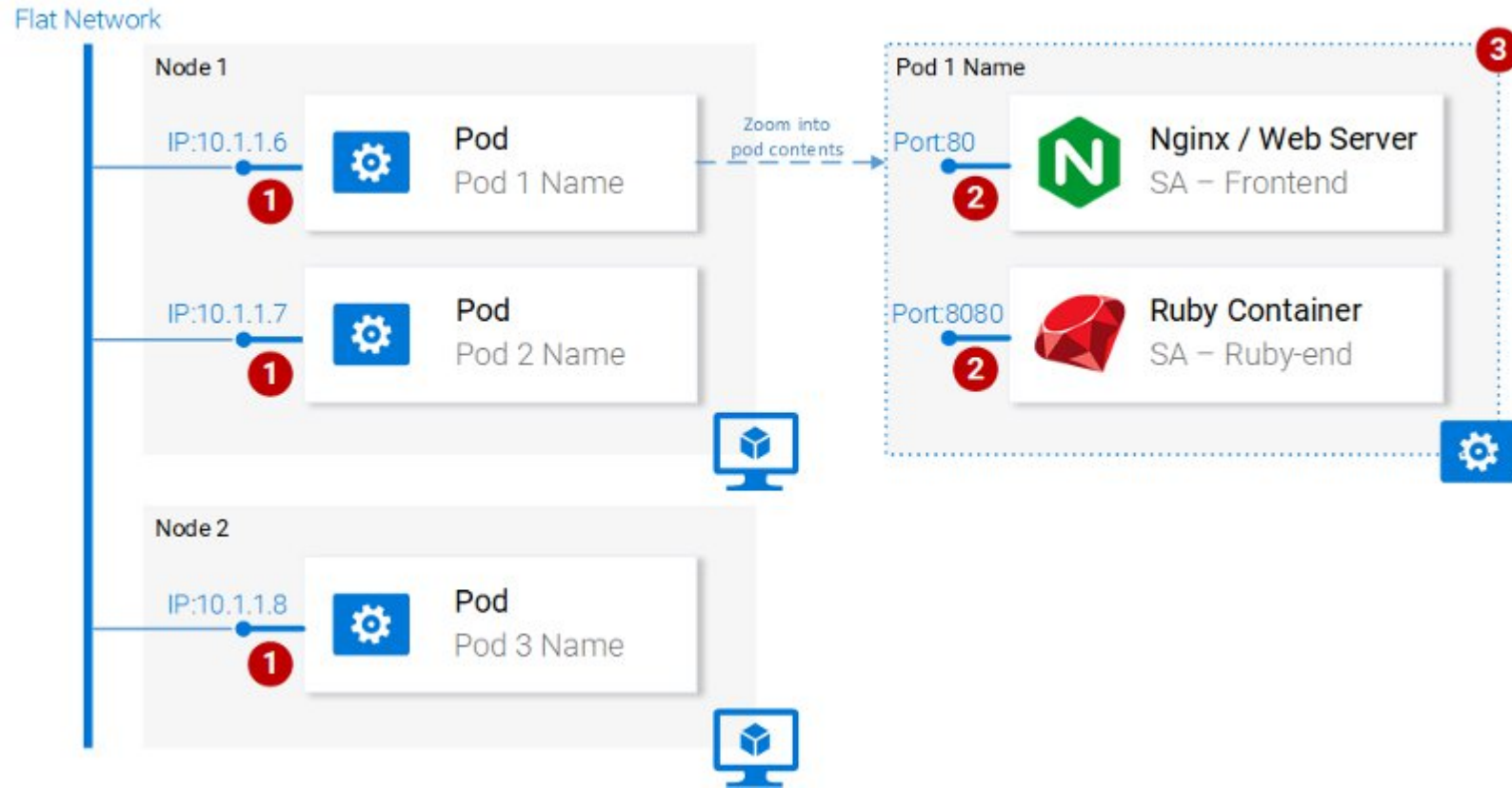
Модули

Почему Kubernetes решил предоставить Pods в качестве наименьшего развертываемого вычислительного модуля? Что делает модуль? Модули могут состоять из одного или даже групп контейнеров, которые используют одну и ту же среду выполнения.

Но действительно ли нам нужно запускать два контейнера в одном модуле? Обычно вы запускаете только один контейнер.

Но в случаях, когда, например, двум контейнерам необходимо совместно использовать тома, или они взаимодействуют друг с другом с помощью межпроцессной связи или иным образом тесно связаны, это становится возможным с помощью модулей.

Еще одна особенность, которую делают возможными модули, заключается в том, что они не привязаны к контейнерам Docker, при желании мы можем использовать другие технологии,



Свойства модуля

- Каждый модуль имеет уникальный IP-адрес в кластере Kubernetes
- Pod может иметь несколько контейнеров. Контейнеры используют одно и то же пространство портов, поэтому они могут обмениваться данными через localhost (по понятным причинам они не могут использовать один и тот же порт), а связь с контейнерами других модулей должна осуществляться совместно с IP-адресом модуля.
- Контейнеры в модуле используют один и тот же том *, один и тот же ip, пространство портов, пространство имен IPC.
- * Контейнеры имеют свои собственные изолированные файловые системы, хотя они могут обмениваться данными с помощью томов ресурсов Kubernetes.

Определение Pod

Ниже у нас есть файл манифеста для pod sa-интерфейса.

- apiVersion: v1
- kind: Pod # 1
- metadata:
- name: sa-frontend # 2
- spec: # 3
- containers:
- - image: rinormaloku/sentiment-analysis-frontend # 4
- name: sa-frontend # 5
- ports: - containerPort: 80 # 6

1. Вид: определяет вид ресурса Kubernetes, который мы хотим создать. В нашем случае, Pod.
2. Name: определяет имя ресурса. Мы назвали его sa-frontend.
3. Спецификация - это объект, который определяет желаемое состояние для ресурса. Наиболее важным свойством спецификации Pods является массив контейнеров.
4. Image - это образ контейнера, который мы хотим запустить в этом модуле.
5. Name - это уникальное имя контейнера в модуле.
6. Порт контейнера: это порт, через который контейнер прослушивает. Это всего лишь индикатор для читателя (удаление порта не ограничивает доступ).

Создание модуля интерфейса SA

Вы можете найти файл для приведенного выше определения модуля в `resource-manifests/sa-frontend-pod.yaml`. Вы либо переходите в своем терминале к этой папке, либо вам придется указать полное местоположение в командной строке. Затем выполните команду:

- `kubectl create -f sa-frontend-pod.yaml`
- `pod "sa-frontend" created`

Чтобы проверить, запущен ли модуль, выполните следующую команду:

- `kubectl get pods`
- | NAME | READY | STATUS | RESTARTS | AGE |
|-------------|-------|---------|----------|-----|
| sa-frontend | 1/1 | Running | 0 | 7s |

Если он все еще находится в режиме создания контейнеров, вы можете выполнить приведенную выше команду с аргументом `--watch`, чтобы обновить информацию, когда модуль находится в рабочем состоянии.

Внешний доступ к приложению

- Для внешнего доступа к приложению мы создаем ресурс Kubernetes типа Service, но для быстрой отладки есть другой вариант, и это перенаправление портов:
- `kubectI port-forward sa-frontend 88:80`
- Forwarding from 127.0.0.1:88 -> 80
- Откройте свой браузер в 127.0.0.1:88, и вы попадете в приложение react.

Способ масштабирования

Одной из основных функций Kubernetes является масштабируемость, чтобы доказать это, давайте запустим еще один модуль.

Для этого создайте другой модульный ресурс со следующим определением:

- **apiVersion: v1**
- **kind: Pod**
- **metadata:**
- **name: sa-frontend2 # The only change**
- **spec:**
- **containers:**
- - **image: rinormaloku/sentiment-analysis-frontend**
- **name: sa-frontend**
- **ports:**
- - **containerPort: 80**

Создайте новый модуль, выполнив следующую команду:

- **kubectl create -f sa-frontend-pod2.yaml**
- **pod "sa-frontend2" created**

Убедитесь, что второй модуль запущен, выполнив:

- **kubectl get pods**
- | NAME | READY | STATUS | RESTARTS | AGE |
|--------------|-------|---------|----------|-----|
| sa-frontend | 1/1 | Running | 0 | 7s |
| sa-frontend2 | 1/1 | Running | 0 | 7s |

Теперь у нас запущены два модуля!

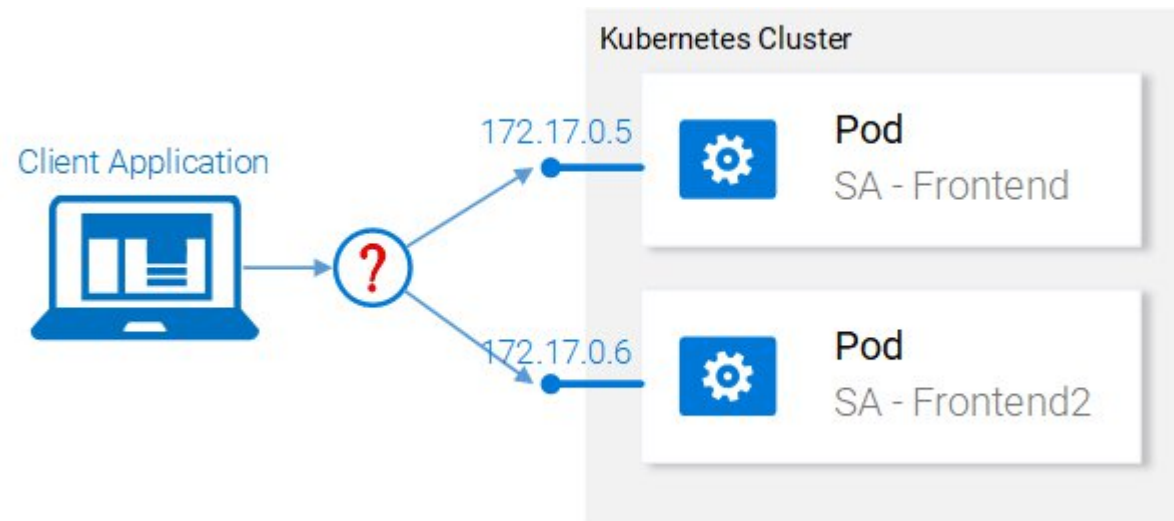
Краткое содержание модуля

Веб-сервер Nginx со статическими файлами работает внутри двух разных модулей. Теперь у нас есть два вопроса:

- Как мы можем предоставить его извне, чтобы сделать его доступным по URL и
- Как мы распределяем нагрузку между ними?

Kubernetes предоставляет нам ресурс Services.

- Рис. ниже - Распределение нагрузки между модулями



Kubernetes на практике — Сервисы

Сервисный ресурс Kubernetes действует как точка входа в набор модулей, которые предоставляют один и тот же функциональный сервис. Этот ресурс выполняет тяжелую работу по поиску сервисов и балансировке нагрузки между ними,

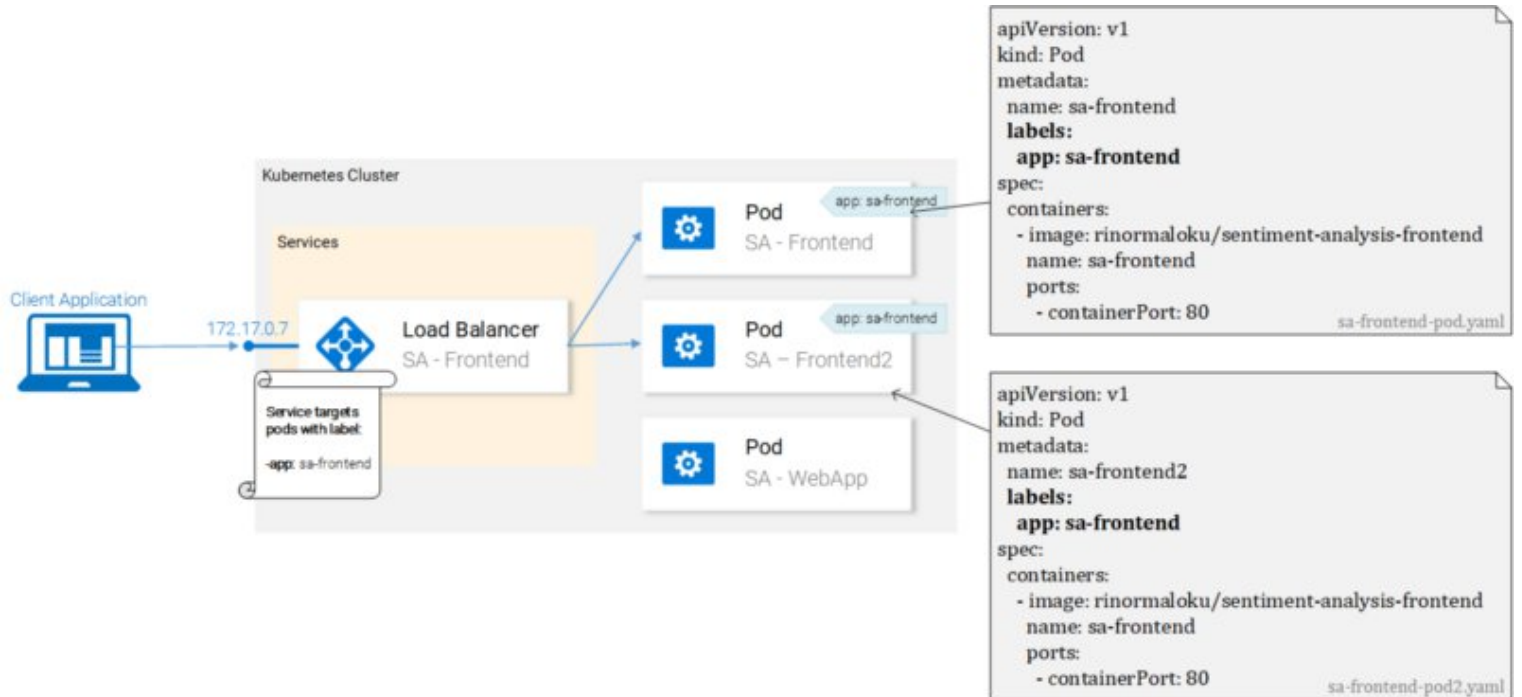
В нашем кластере Kubernetes у нас будут модули с различными функциональными сервисами. (Интерфейс, веб-приложение Spring и приложение Flask Python).

Итак, возникает вопрос, как служба узнает, на какие модули настроить таргетинг? Как она генерирует список конечных точек для модулей?

Это делается с помощью меток, и это двухэтапный процесс:

1. Нанесение ярлыка на все модули, на которые мы хотим настроить таргетинг нашего сервиса, и
2. Применение “селектора” к нашему сервису, чтобы определить, на какие помеченные модули следует ориентироваться.

Мы видим, что модули помечены как “app: sa-frontend”, и сервис ориентирован на модули с этим ярлыком.



Метки

Ярлыки предоставляют простой метод организации ваших ресурсов Kubernetes. Они представляют собой пару ключ-значение и могут быть применены к любому ресурсу. Измените манифесты для модулей, чтобы они соответствовали примеру, показанному ранее на рисунке 17.

Сохраните файлы после завершения изменений и примените их с помощью следующей команды:

- `kubectl apply -f sa-frontend-pod.yaml`
- Warning: kubectl apply should be used on resource created by either kubectl create --save-config or kubectl apply
- pod "sa-frontend" configured
- `kubectl apply -f sa-frontend-pod2.yaml`
- Warning: kubectl apply should be used on resource created by either kubectl create --save-config or kubectl apply
- pod "sa-frontend2" configured

Мы получили предупреждение (применить вместо создать, понял). Во второй строке мы видим, что модули "sa-frontend" и "sa-frontend2" настроены. Мы можем убедиться, что модули были помечены, отфильтровав модули, которые мы хотим отобразить:

```
kubectl get pod -l app=sa-frontend
```

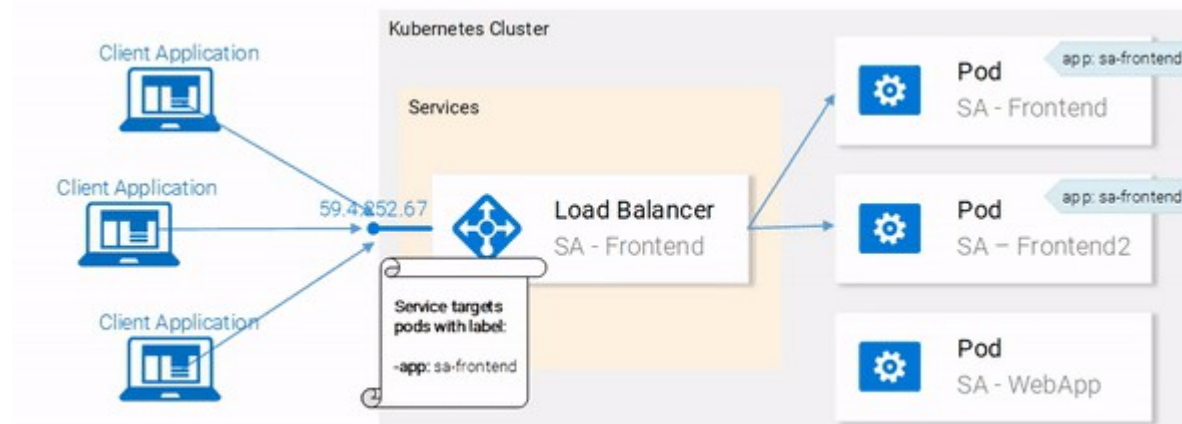
NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

sa-frontend	1/1	Running	0	2h
-------------	-----	---------	---	----

sa-frontend2	1/1	Running	0	2h
--------------	-----	---------	---	----

Другой способ проверить, что наши модули помечены, - это добавить флаг `--show-labels` к приведенной выше команде. При этом будут отображены все метки для каждого модуля.

Модули помечены, и мы готовы настроить таргетинг на них с помощью нашего сервиса. Давайте начнем определять службу типа LoadBalancer, показанную на рис.



Определение сервиса

Определение службы балансировки нагрузки на языке YAML показано ниже:

- `apiVersion: v1`
- `kind: Service` # 1
- `metadata:`
- `name: sa-frontend-lb`
- `spec:`
- `type: LoadBalancer` # 2
- `ports:`
- `- port: 80` # 3
- `protocol: TCP` # 4
- `targetPort: 80` # 5
- `selector:` # 6
- `app: sa-frontend` # 7

1. Вид: сервис.
2. Тип: Тип спецификации, мы выбираем `LoadBalancer`, потому что хотим сбалансировать нагрузку между блоками.
3. Порт: Указывает порт, через который служба получает запросы.
4. Протокол: определяет связь.
5. `targetPort`: порт, на который пересылаются входящие запросы.
6. Селектор: объект, содержащий свойства для выбора модулей.
7. `app: sa-frontend` Определяет, на какие модули следует ориентироваться, только на модули, помеченные как `"app: sa-frontend"`.

Чтобы создать службу, выполните следующую команду:

1. `kubectl create -f service-sa-frontend-lb.yaml`
2. `service "sa-frontend-lb" created`

Kubernetes на практике — развертывания

Развертывания Kubernetes помогают нам с одной константой в жизни каждого приложения, и это изменение.

Ресурс развертывания автоматизирует процесс перехода от одной версии приложения к следующей с нулевым временем простоя и в случае сбоев позволяет нам быстро вернуться к предыдущей версии.

Развертывания на практике

В настоящее время у нас есть два модуля и сервис, предоставляющий их и балансирующий нагрузку между ними (см. рис. 19.). Мы упоминали, что раздельное развертывание модулей далеко от совершенства.

Для этого требуется отдельное управление каждым из них (создание, обновление, удаление и мониторинг их работоспособности). О быстрых обновлениях и откатах не может быть и речи! Это неприемлемо, и ресурс Kubernetes для развертывания решает каждую из этих проблем.

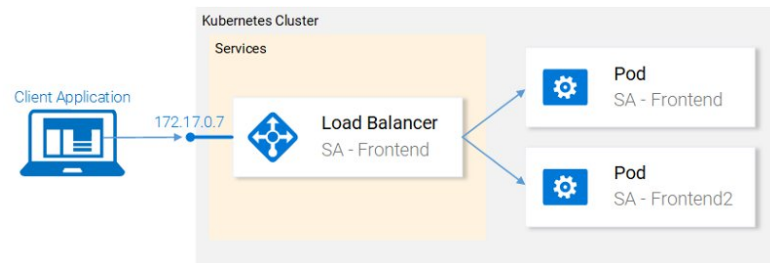


Рис. Текущее состояние

Прежде чем мы продолжим, давайте сформулируем, чего мы хотим достичь, поскольку это даст нам обзор, который позволит нам понять определение манифеста для ресурса развертывания.

Чего мы хотим, так это:

- Два модуля изображения `ginormaloku`/анализ настроек-интерфейс
- Развертывания с нулевым временем простоя,
- Модули, помеченные `app: sa-frontend`, чтобы службы были обнаружены службой `sa-frontend-lb`.

Определение развертывания

- Определение ресурса YAML, которое обеспечивает все вышеупомянутые моменты:

- `apiVersion: apps/v1`
- `kind: Deployment` # 1
- `metadata:`
- `name: sa-frontend`
- `spec:`
- `selector:` # 2
 - `matchLabels: app: sa-frontend`
- `replicas: 2` # 3
- `minReadySeconds: 15`
- `strategy:`
- `type: RollingUpdate` # 4
- `rollingUpdate: maxUnavailable: 1` # 5
- `maxSurge: 1` # 6
- `template:` # 7
 - `metadata:`
 - `labels:`
 - `app: sa-frontend` # 8
 - `spec:`
 - `containers:`
 - `- image: rinormaloku/sentiment-analysis-frontend`
 - `imagePullPolicy: Always` # 9
 - `name: sa-frontend`
 - `ports:` - `containerPort: 80`

1. Вид: развертывание.
2. Селектор: модули, соответствующие селектору, будут переданы под управление этого развертывания.
3. Replicas - это свойство объекта deployments Spec, которое определяет, сколько модулей мы хотим запустить. Итак, только 2.
4. Тип определяет стратегию, используемую в данном развертывании при переходе от текущей версии к следующей. Стратегия RollingUpdate гарантирует отсутствие простоев при развертывании.
5. maxUnavailable - это свойство объекта RollingUpdate, которое определяет максимально допустимое количество недоступных модулей (по сравнению с желаемым состоянием) при выполнении текущего обновления. Для нашего развертывания, в котором есть 2 реплики, это означает, что после завершения работы одного модуля у нас все равно будет запущен один модуль, таким образом сохраняя доступность нашего приложения.
6. maxSurge - это еще одно свойство объекта RollingUpdate, которое определяет максимальное количество модулей, добавленных в развертывание (по сравнению с желаемым состоянием). Для нашего развертывания это означает, что при переходе на новую версию мы можем добавить один модуль, который добавляет до 3 модулей одновременно.
7. Template: указывает шаблон модуля, который развертывание будет использовать для создания новых модулей. Скорее всего, сходство с Pods сразу бросилось вам в глаза.
8. app: sa-frontend ярлык, который будет использоваться для модулей, созданных с помощью этого шаблона.
9. imagePullPolicy если установлено значение Всегда, оно будет извлекать изображения контейнеров при каждом повторном использовании.

Активация развертывания

- `kubectl apply -f sa-frontend-deployment.yaml`
- deployment "sa-frontend" created
- Проверка:
- `kubectl get pods`

NAME	READY	STATUS	RESTARTS	AGE
sa-frontend	1/1	Running	0	2d
sa-frontend-5d5987746c-ml6m4	1/1	Running	0	1m
sa-frontend-5d5987746c-mzsgg	1/1	Running	0	1m
sa-frontend2	1/1	Running	0	2d

У нас есть 4 работающих модуля, два модуля, созданных при развертывании, а два других - те, которые мы создали вручную.

Удалите те, которые мы создали вручную с помощью команды `kubectl delete pod <pod-name>`.

Упражнение: удалите также один из модулей развертывания и посмотрите, что произойдет. Подумайте о причине, прежде чем читать объяснение ниже.

Объяснение: При удалении одного модуля развертывание заметило, что текущее состояние (запущен 1 модуль) отличается от желаемого состояния (запущено 2 модуля), поэтому оно запустило другой модуль.

Итак, что было такого хорошего в развертываниях, помимо сохранения желаемого состояния?

Преимущества развертывания

- Преимущество № 1: развертывание с нулевым временем простоя

За кулисами “RollingUpdate”

После того, как мы применили новое развертывание, Kubernetes сравнивает новое состояние со старым. В нашем случае новое состояние запрашивает два модуля с изображением, `rinormaloku/sentiment-analysis-frontend:green`. Это отличается от текущего состояния, поэтому запускается RollingUpdate.

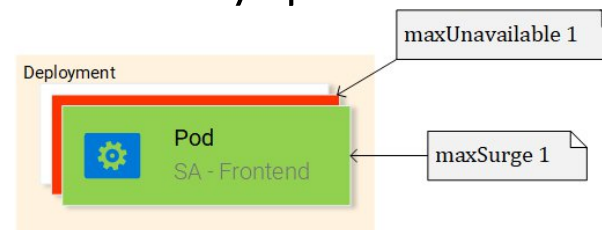


Рис. RollingUpdate, заменяющий модули

RollingUpdate действует в соответствии с указанными нами правилами, такими как "maxUnavailable: 1" и "maxSurge: 1". Это означает, что развертывание может завершить только один модуль и запустить только один новый модуль. Этот процесс повторяется до тех пор, пока все модули не будут заменены

Преимущество №2: быстрый откат

```
kubectl rollout undo deployment sa-frontend --to-revision=1
```

```
deployment "sa-frontend" rolled back
```

Вы обновляете страницу, и изменение отменяется!

Kubernetes и все остальное на практике

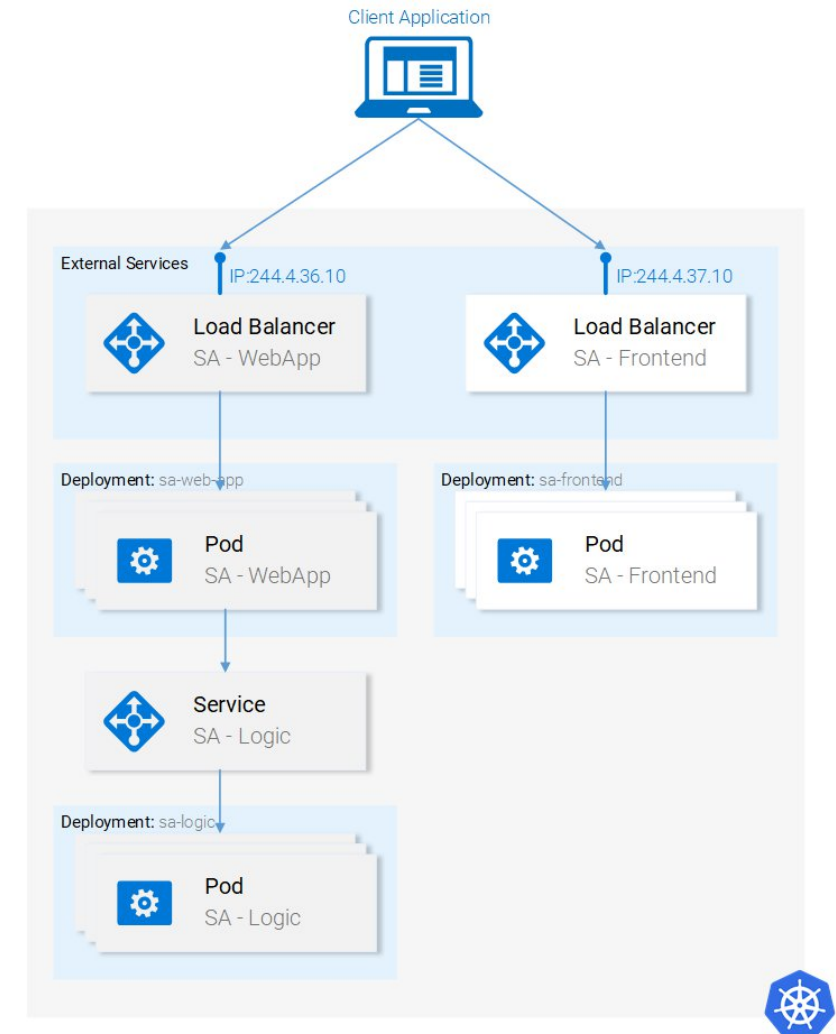
SA-Logic развертывания

Перейдите в вашем терминале к папке resource-manifests и выполните следующую команду:

```
kubectl apply -f sa-logic-deployment.yaml --record  
deployment "sa-logic" created
```

SA-Logic развертывания создала три модуля.
(Запуск контейнера нашего приложения на Python).
Он пометил их с помощью app: sa-logic. этой маркировки позволяет нам настроить на них таргетинг с помощью селектора из службы SA-Logic.
(файл sa-logic-deployment.yaml)

Все те же концепции используются снова, давайте сразу перейдем к следующему ресурсу: службе SA-Logic.



Логика Service SA

Java-приложение (работающее в пакетах развертывания SA — WebApp) зависит от анализа настроек, выполняемого приложением Python.

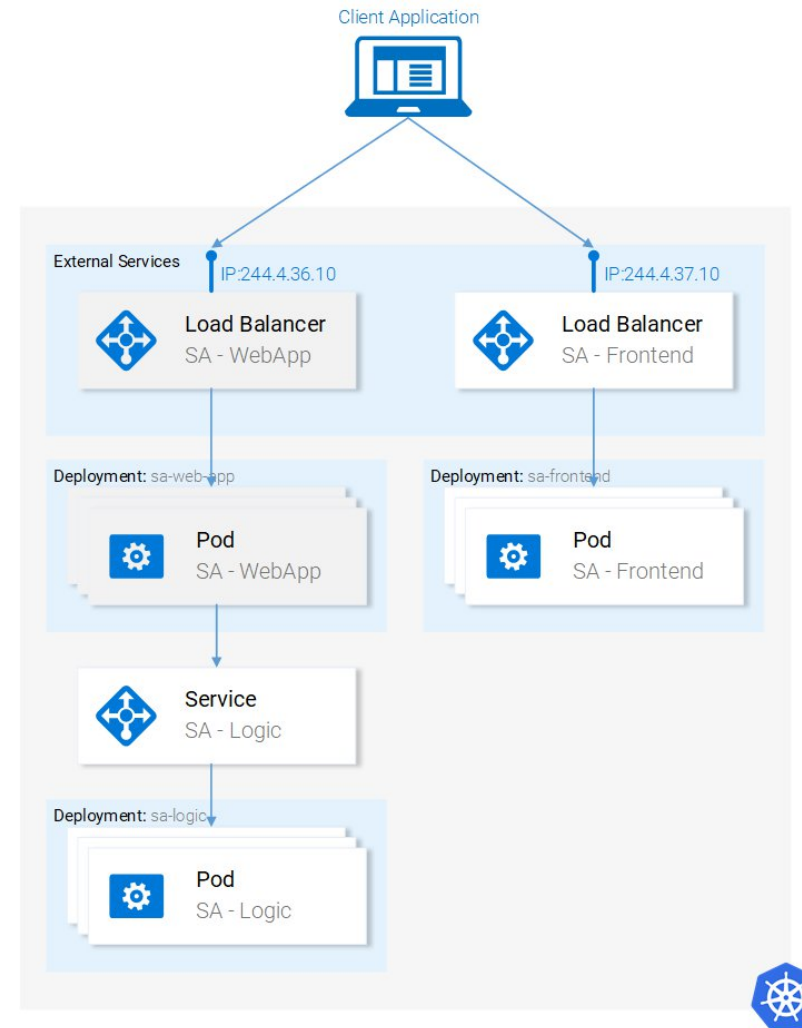
Но теперь, в отличие от того времени, когда мы запускали все локально, у нас нет одного приложения Python, прослушивающего один порт, у нас есть 2 модуля, и при необходимости мы могли бы использовать больше.

Вот почему нам нужен сервис, который “действует как точка входа в набор модулей, предоставляющих один и тот же функциональный сервис”. Это означает, что мы можем использовать службу SA-Logic в качестве точки входа для всех модулей SA-Logic.

Давайте сделаем это:

- `kubectl apply -f service-sa-logic.yaml`
- `service "sa-logic" created`

Обновлено состояние приложения: У нас запущены 2 модуля (содержащие приложение Python), и у нас есть служба SA-Logic, действующая как точка входа, которую мы будем использовать в модулях SA-WebApp.



Развертывание SA-WebApp

Есть еще одна особенность. Если вы откроете файл `sa-web-app-deployment.yaml`, вы найдете эту часть новой:

- `- image: rinormaloku/sentiment-analysis-web-app`
- `imagePullPolicy: Always`
- `name: sa-web-app`
- `env:`
 - `- name: SA_LOGIC_API_URL`
 - `value: "http://sa-logic"`
- `ports:`
 - `- containerPort: 8080`

Первое, что нас интересует, это то, что делает свойство `env` ? И мы предполагаем, что он объявляет переменную среды `SA_LOGIC_API_URL` со значением `"http://sa-logic"` внутри наших модулей. Но почему мы инициализируем его `http://sa-logic` , что такое `sa-logic`?

KUBE-DNS

- В Kubernetes есть специальный модуль **kube-dns**. И по умолчанию все модули используют его в качестве DNS-сервера. Одним из важных свойств **kube-dns** является то, что он создает запись DNS для каждой созданной службы.
- Это означает, что при создании службы **sa-logic** она получила IP-адрес. Его имя было добавлено в качестве записи (вместе с IP) в **kube-dns**. Это позволяет всем модулям преобразовывать **sa-logic** в IP-адрес служб SA-Logic services.

Развертывание веб-приложения SA (продолжение)

Выполните команду:

- `kubectl apply -f sa-web-app-deployment.yaml --record`
- `deployment "sa-web-app" created`

Выполнено. Нам остается предоставить доступ к пакетам SA-WebApp извне, используя службу балансировки нагрузки.

Это позволяет нашему приложению react отправлять http-запросы к сервису, который действует как точка входа в модули SA-WebApp.

Service SA-WebApp

Откройте файл `service-sa-web-app-lb.yaml`

Выполните команду:

- `kubectl apply -f service-sa-web-app-lb.yaml`
- `service "sa-web-app-lb" created`

Есть один единственный диссонанс.

Когда мы развертывали модули SA-Frontend, изображение нашего контейнера указывало на наше SA-WebApp в `http://localhost:8080/sentiment`.

Но теперь нам нужно обновить его, чтобы указать IP-адрес SA-WebApp Loadbalancer. (Который действует как точка входа в модули SA-WebApp).

Точка входа в модули SA-WebApp

1. Получите IP-адрес SA-WebApp Loadbalancer, выполнив следующую команду:

```
minikube service list
```

```
|-----|-----|-----|
| NAMESPACE | NAME       | URL                |
|-----|-----|-----|
| default   | kubernetes | No node port       |
| default   | sa-frontend-lb | http://192.168.99.100:30708 |
| default   | sa-logic     | No node port       |
| default   | sa-web-app-lb | http://192.168.99.100:31691 |
| kube-system | kube-dns     | No node port       |
| kube-system | kubernetes-dashboard | http://192.168.99.100:30000 |
|-----|-----|-----|
```

2. Используйте IP-адрес SA-WebApp LoadBalancer в файле sa-frontend/src/App.js, как показано ниже:

```
analyzeSentence() {
  fetch('http://192.168.99.100:31691/sentiment', { /* shortened for brevity */ })
    .then(response => response.json())
    .then(data => this.setState(data));
}
```

3. Создайте статические файлы npm run build (вам нужно перейти к папке sa-frontend)

Точка входа в модули SA-WebApp (продолжение)

4. Создайте образ контейнера:

- `docker build -f Dockerfile -t $DOCKER_USER_ID/sentiment-analysis-frontend:minikube .`

5. Отправьте изображение в Docker hub.

- `docker push $DOCKER_USER_ID/sentiment-analysis-frontend:minikube`

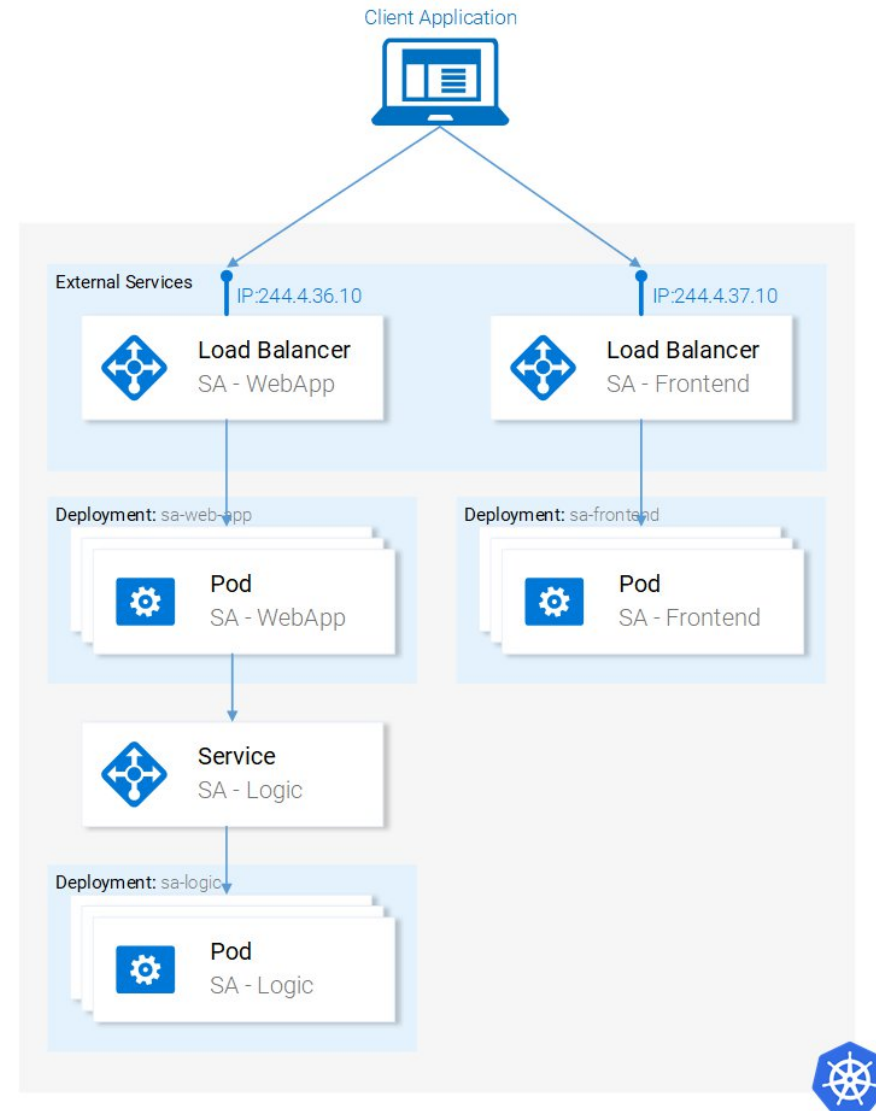
6. Отредактируйте `sa-frontend-deployment.yaml` для использования нового образа и

7. Выполните команду `kubectl apply -f sa-frontend-deployment.yaml`

Обновите браузер или, если вы закрыли окно, выполните

- `minikube service sa-frontend-lb.`

Попробуйте, введя предложение!



Литература

- <https://www.freecodecamp.org/news/learn-kubernetes-in-under-3-hours-a-detailed-guide-to-orchestrating-containers-114ff420e882>
- <https://github.com/rinormaloku/k8s-mastery>
- **Kubernetes в действии** [Марко Лукса](#) ASIN: 1617293725
Издатель : Мэннинг; 1-е издание (20 января 2018)
Язык: Английский 624 с. ISBN-10: 9781617293726